

Singra Vault — Phasensteuerung für KI-Agenten

Zweck dieses Dokuments

Dieses Dokument ist **kein Ersatz** für das detaillierte Architekturkonzept „Singra Vault — Neues Integrations-, Quarantäne- und Manipulationsschutzsystem“.

Das Architekturkonzept bleibt die fachliche und sicherheitstechnische Grundlage. Dieses Dokument dient nur dazu, die Umsetzung in klar getrennte, kontrollierbare Phasen aufzuteilen, damit ein KI-Modell immer nur einen begrenzten Auftrag erhält und nicht versehentlich das gesamte System gleichzeitig umbaut.

Die Grundregel lautet:

Ein KI-Agent arbeitet immer nur an genau einer Phase. Alles außerhalb dieser Phase ist verboten, außer es wird ausdrücklich als Vorprüfung, Test oder Dokumentation verlangt.

1. Gesamtstatus

Bereits umgesetzt

Phase 0 — Architektur, ADR und Inventar

Status: **abgeschlossen**

Ziel dieser Phase war es, das neue Modell normativ festzuhalten und die alten Pfade zu inventarisieren.

Erwartete Ergebnisse:

- ADR für das neue Operation-Log-/Record-Integrity-Modell.
- Inventar alter Vault-, Sync-, Quarantäne-, Integrity- und Rebaseline-Pfade.
- Phasenplan für spätere Entfernung alter Logik.
- Keine Produktionspfade angebunden.
- Keine UI-Änderungen.
- Keine Migration aktiver Nutzerdaten.

Tatsächlich gemeldete Ergebnisse:

- `docs/adr/0004-vault-operation-log-and-record-integrity.md`
- `docs/vault-op-log/inventory-and-phase-plan.md`
- `src/services/vaultOpLog/README.md`

Bewertung:

- Entspricht dem Plan.
- Sicherheitsrisiko gering, weil nur Dokumentation und Inventar.

Phase 1 — Reine Kryptografie-, Hash-, AAD-, Signatur- und Trust-Module

Status: **abgeschlossen**

Ziel dieser Phase war es, reine, testbare Grundbausteine zu erstellen, ohne sie in die Produktivpfade einzubauen.

Erwartete Ergebnisse:

- zentrale Canonicalization,
- AAD-Erzeugung,
- Hash-Funktionen,
- Record-Seal/Open-Logik,
- Operation-Signing,
- Device-Trust-Klassifikation,
- Unit-Tests,
- keine Produktionseinbindung.

Tatsächlich gemeldete Ergebnisse:

- `src/services/vaultOpLog/types.ts`
- `src/services/vaultOpLog/canonicalJson.ts`
- `src/services/vaultOpLog/recordAad.ts`
- `src/services/vaultOpLog/recordHashes.ts`
- `src/services/vaultOpLog/cryptoRecordService.ts`
- `src/services/vaultOpLog/operationSigningService.ts`
- `src/services/vaultOpLog/deviceTrustService.ts`
- `src/services/vaultOpLog/index.ts`
- Unit-Tests unter `src/services/vaultOpLog/__tests__`

Wichtige Sicherheitsgrenzen:

- Keine UI-Anbindung.
- Keine Autofill-Anbindung.
- Keine Export-Anbindung.
- Keine alten Pfade entfernen.
- Keine Migration ausführen.
- Keine Supabase-Schreibpfade verwenden.

Bewertung:

- Entspricht dem Plan.
- Gute Trennung, weil die neuen Module noch nicht produktiv genutzt werden.

Phase 2 — Supabase-Schema, RLS und RPC-Grundlage

Status: **abgeschlossen, mit Hardening-Korrekturen**

Ziel dieser Phase war es, die serverseitige Speicher- und Transportstruktur für Records, Operationen, Trust Records und Vault Heads anzulegen.

Erwartete Ergebnisse:

- `vault_records`,
- `vault_operations`,
- `vault_device_trust_records`,
- `vault_op_log_heads`,
- RLS,
- direkte Writes verhindern,
- zentrale RPCs,
- statische Vertragstests,
- keine Client-Runtime-Anbindung.

Gemeldete Ergebnisse:

- `supabase/migrations/20260504130000_vault_op_log_phase2_records_operations_trust.sql`
- `supabase/migrations/20260504130100_vault_op_log_phase2_rpcs.sql`
- `src/test/vault-op-log-phase2-migration-contract.test.ts`
- `src/test/integration/vault-op-log-phase2-integration.test.ts`

Nachträgliche Hardening-Korrekturen:

- `previous_record_hash` wurde zu `previous_ciphertext_hash` korrigiert.
- CAS prüft jetzt gegen `vault_records.ciphertext_hash`.
- `intent_id` und `rebased_from_op_id` wurden für das Rebase-Modell ergänzt.
- `bootstrap_vault_trust` wurde als eigener einmaliger Trust-Bootstrap-RPC ergänzt.
- SECURITY-DEFINER-RPCs wurden auf `search_path`, Grants und dynamische SQL geprüft.
- JSONB-Roundtrip-Testvektoren wurden ergänzt.

Bewertung:

- Entspricht dem Plan nach Korrektur.
- Wichtig: Phase 2 ist weiterhin noch keine Produktivumstellung.
- Das alte System läuft noch; neue Tabellen/RPCs sind vorbereitet.

2. Noch offene Hauptphasen

Die folgenden Phasen sollten einzeln an ein KI-Modell gegeben werden.

Jede Phase enthält:

- Ziel,
- erlaubte Arbeiten,
- verbotene Arbeiten,
- Inputs,
- erwartete Outputs,

- Tests,
 - Abbruchkriterien,
 - Review-Fragen für dich als Nicht-Programmierer.
-

Phase 3 — Repository-Layer und Feature-Flag, ohne UI

Ziel

Phase 3 baut die technische Client-Schnittstelle zu den neuen Supabase-RPCs und Tabellen. Sie verbindet die neuen reinen Module aus Phase 1 mit den RPCs aus Phase 2, aber noch nicht mit der normalen UI, Autofill, Export oder dem alten VaultContext.

Phase 3 ist die erste Phase mit echter Client-Integration, aber sie bleibt hinter einem Feature-Flag und darf keine bestehenden Produktivpfade ersetzen.

Hauptziel in einem Satz

Baue einen isolierten `vaultOpLog`-Repository-/Sync-Zugriff, der RPCs typisiert aufruft, Operationen sicher serialisiert/deserialisiert und DB-Roundtrips gegen Canonicalization testet — ohne UI- oder Runtime-Umbau.

Erlaubte Arbeiten

Der Agent darf:

- Repository-Service für Supabase-RPCs erstellen.
- Typisierte Mapper zwischen DB-Rows und Domain-Typen erstellen.
- `submit_vault_operation` kapseln.
- `get_vault_head` kapseln.
- `get_vault_changes_since` kapseln.
- `get_vault_records_by_ids` kapseln.
- `bootstrap_vault_trust` kapseln.
- Feature-Flag für neuen OpLog-Pfad einführen.
- Tests für RPC-Request-/Response-Mapping schreiben.
- Tests für JSONB/canonical signed_body roundtrip schreiben.
- Tests für idempotente Retry-Semantik schreiben.
- Tests für stale head / stale record / conflict result schreiben.
- Integrationstest-Skelett erweitern, aber nur gated über lokale Supabase-Testumgebung.

Verbotene Arbeiten

Der Agent darf nicht:

- UI ändern.
- VaultContext produktiv umbauen.

- Autofill ändern.
- Export ändern.
- Suche ändern.
- Clipboard ändern.
- alte Runtime-Sicherheitslogik entfernen.
- alte Vault-Daten migrieren.
- neue OpLog-Daten in der echten Vault-Liste anzeigen.
- direkte Writes auf alte Tabellen einbauen.
- alte Tabellen als Trust-Quelle verwenden.
- Feature-Flag umgehen.
- Tests löschen, nur weil sie fehlschlagen.

Pflichtdateien oder wahrscheinliche Dateien

Mögliche neue Dateien:

- `src/services/vaultOpLog/vaultOpLogRepository.ts`
- `src/services/vaultOpLog/vaultOpLogRpcTypes.ts`
- `src/services/vaultOpLog/vaultOpLogMappers.ts`
- `src/services/vaultOpLog/vaultOpLogFeatureFlags.ts`
- `src/services/vaultOpLog/__tests__/vaultOpLogRepository.test.ts`
- `src/services/vaultOpLog/__tests__/vaultOpLogMappers.test.ts`

Bestehende Dateien, die eventuell ergänzt werden dürfen:

- `src/services/vaultOpLog/index.ts`
- `src/services/vaultOpLog/types.ts`
- Phase-2-Integrationstestdatei, falls bereits angelegt.

Wichtige Sicherheitsregeln

- `signed_body` aus DB darf nie blind als „signiert gültig“ gelten.
- Der Client muss canonicalize erneut ausführen.
- `op_hash` muss nach Roundtrip identisch bleiben.
- Response-Mapping darf keine sicherheitsrelevanten Felder verlieren.
- `previous_ciphertext_hash`, `intent_id`, `rebased_from_op_id`, `base_vault_head`, `resulting_vault_head` müssen vollständig erhalten bleiben.
- Repository-Layer darf keine Sicherheitsentscheidung treffen, die in die State Machine gehört.
- Repository-Layer darf nur transportieren, mappen, validieren und Fehler klassifizieren.

Erwartete Tests

Mindestens:

- Mapper behalten alle sicherheitsrelevanten Felder.
- `signed_body` survives JSONB-like roundtrip.
- `op_hash` bleibt stabil.
- gleicher `op_id` + gleicher `op_hash` wird als idempotent behandelt.
- gleicher `op_id` + anderer `op_hash` wird als Fehler klassifiziert.
- stale `base_vault_head` wird als Rebase-erforderlich klassifiziert, nicht als Manipulation.

- stale `previous_ciphertext_hash` wird als Record-Konflikt klassifiziert.
- Bootstrap-RPC wird nur als Initialisierungspfad gekapselt.

Akzeptanzkriterien

Phase 3 ist nur fertig, wenn:

- neue Repository-Schicht existiert,
- alle RPCs typisiert gekapselt sind,
- Feature-Flag existiert,
- keine UI/Autofill/Export/Suche/Clipboard-Anbindung erfolgt ist,
- keine alten Runtime-Pfade entfernt wurden,
- alle neuen Tests grün sind,
- bestehende bekannte Fehlschläge klar als vorbestehend markiert sind,
- Review-Zusammenfassung erstellt wurde.

Review-Fragen für dich

Frage den Agenten am Ende:

1. Welche Dateien wurden geändert?
2. Wurde irgendein UI-, VaultContext-, Autofill-, Export- oder Suchpfad verändert?
3. Ist alles hinter Feature-Flag?
4. Wie wird `signed_body` nach DB-Roundtrip verifiziert?
5. Wie wird stale `base_vault_head` behandelt?
6. Wie wird stale `previous_ciphertext_hash` behandelt?
7. Welche Tests beweisen das?

Wenn eine Antwort unklar ist, nicht weiter zu Phase 4 gehen.

Phase 4 — Lokale Pending Queue, Operation Builder und Rebase-Workflow

Ziel

Phase 4 baut die lokale Arbeitslogik für neue Operationen: Erzeugen, Signieren, Speichern, Retry und Rebase.

Diese Phase darf noch immer keine normale UI ersetzen. Sie baut die Mechanik, mit der spätere Vault-Änderungen als signierte Operationen entstehen.

Hauptziel in einem Satz

Baue die lokale Operationserzeugung und Pending-Queue so, dass Nutzeraktionen als signierte Operationen mit stabiler Intent-ID, idempotentem Retry und sicherem Rebase-Modell verarbeitet werden können.

Erlaubte Arbeiten

Der Agent darf:

- Operation Builder erstellen.
- Pending Operation Store erstellen.
- Local queue persistence implementieren.
- Retry-Logik implementieren.
- Rebase bei stale Vault Head implementieren.
- `intent_id` stabil pro Nutzerintent verwalten.
- `rebased_from_op_id` setzen.
- neue `op_id` bei Rebase erzeugen.
- alte stale Operation als ersetzt markieren.
- Tests für Crash-/Retry-Verhalten schreiben.
- Tests für Multi-Tab-Locking vorbereiten oder implementieren.

Verbotene Arbeiten

Der Agent darf nicht:

- bestehende Vault-Speicherlogik ersetzen.
- UI produktiv auf neue Queue umstellen.
- alte Mutation Queue löschen.
- alte Offline-Sync-Logik entfernen.
- Daten migrieren.
- Autofill/Export/Suche ändern.

Wichtige Sicherheitsregeln

- Retry derselben Operation nutzt dieselbe `op_id`.
- Rebase derselben Nutzerabsicht nutzt dieselbe `intent_id`, aber neue `op_id`.
- Rebase muss neu signieren.
- Rebase darf nur passieren, wenn der betroffene Record-CAS noch gültig ist oder sauber als Konflikt klassifiziert wurde.
- Eine stale Operation darf nicht blind erneut gesendet werden, wenn `base_vault_head` nicht mehr aktuell ist.
- Keine Operation darf ohne Signatur in die Queue.

Erwartete Tests

Mindestens:

- Operation Builder erzeugt vollständige signed bodies.
- `intent_id` bleibt über Rebase stabil.
- `op_id` bleibt über Retry stabil.
- Rebase erzeugt neue `op_id`.
- `rebased_from_op_id` zeigt auf alte Operation.
- stale head führt zu Rebase-needed.
- stale record hash führt zu Conflict.
- Queue bleibt nach simuliertem Absturz konsistent.

- keine nackten Payloads ohne Signatur in Queue.

Akzeptanzkriterien

Phase 4 ist nur fertig, wenn:

- Pending Queue existiert,
- Operation Builder existiert,
- Rebase-Regeln technisch umgesetzt sind,
- Retry und Rebase sauber getrennt sind,
- keine UI-Produktivpfade angebunden sind,
- Tests Rebase/Retry/Conflict beweisen.

Review-Fragen für dich

1. Was ist der Unterschied zwischen Retry und Rebase?
 2. Bleibt `intent_id` gleich?
 3. Wird bei Rebase neu signiert?
 4. Wird bei stale `previous_ciphertext_hash` ein Konflikt erzeugt?
 5. Gibt es Tests dafür?
-

Phase 5 — Vault State Machine und Verifikationspipeline

Ziel

Phase 5 baut das Herzstück des neuen Systems: Die lokale Zustandsmaschine, die Operationen und Records verifiziert, klassifiziert und in `verified`, `conflict`, `quarantine`, `safeMode` oder `lockedCritical` einordnet.

Hauptziel in einem Satz

Implementiere die State Machine, die Remote-Operationen und Records lokal kryptografisch prüft und erst danach entscheidet, ob Daten verifiziert, quarantänisiert, als Konflikt markiert oder in Safe Mode geführt werden.

Erlaubte Arbeiten

Der Agent darf:

- `vaultStateMachine` implementieren.
- `verifyOperation` Pipeline implementieren.
- `verifyRecordContext` implementieren.
- `applyRemoteOperation` implementieren.
- `applyTrustedDelete` implementieren.
- Konflikterkennung implementieren.

- Quarantäneklassifikation implementieren.
- Safe-Mode-Auslöser vorbereiten.
- lockedCritical-Kriterien implementieren.
- Tests für Manipulation/Fork/Rollback/Unknown Device schreiben.

Verbotene Arbeiten

Der Agent darf nicht:

- UI anbinden.
- Autofill anbinden.
- Export anbinden.
- Suche anbinden.
- alte Baseline-Logik entfernen.
- Migration ausführen.
- Quarantäne-UI umbauen.

Sicherheitsinvarianten

- Kein Record wird entschlüsselt, bevor Operation, Autor, Signatur, AAD und CiphertextHash geprüft sind.
- Unknown-author Records werden nicht entschlüsselt.
- Invalid-signature Records werden nicht entschlüsselt.
- AAD mismatch wird nicht entschlüsselt.
- CiphertextHash mismatch wird nicht entschlüsselt.
- Konflikte gültiger Geräte sind keine Manipulation.
- Kategoriefehler blockieren nicht den gesamten Vault.
- lockedCritical nur bei Root-Problemen.

Erwartete Tests

Mindestens:

- gültige Operation + gültiger Record → verified.
- unbekanntes Gerät → quarantinedUnknownAuthor, kein decrypt.
- ungültige Signatur → quarantinedTampered, kein decrypt.
- AAD mismatch → kein decrypt.
- ciphertextHash mismatch → kein decrypt.
- gültige Delete-Operation → deletedByTrustedDevice.
- fehlender Record ohne Delete → quarantinedMissingWithoutDelete.
- zwei gültige Updates gleicher Basis → conflict.
- Kategorie manipuliert → Kategorie quarantined, Vault nicht lockedCritical.
- Manifest kaputt → lockedCritical.
- Operation-Log-Lücke → safeMode oder safeMode recommended.

Akzeptanzkriterien

Phase 5 ist nur fertig, wenn:

- State Machine existiert,

- alle oben genannten Zustände testbar sind,
- keine UI-Anbindung erfolgt ist,
- keine Quarantäne-Records entschlüsselt werden,
- Tests beweisen, dass alte Fehler nicht wieder entstehen.

Review-Fragen für dich

1. Wo ist die Funktion, die entscheidet, ob ein Record entschlüsselt werden darf?
 2. Welche Tests beweisen, dass manipulierte Records nicht entschlüsselt werden?
 3. Welche Tests beweisen, dass Kategoriefehler den Vault nicht sperren?
 4. Welche Tests beweisen, dass legitime Konflikte keine Quarantäne sind?
-

Phase 6 — Trusted Snapshots und Recovery-Operationen

Ziel

Phase 6 baut lokale signierte Snapshots und Recovery-Aktionen. Snapshots sind nicht die Wahrheit, sondern Wiederherstellungspunkte.

Hauptziel in einem Satz

Implementiere Trusted Snapshots als lokale, verschlüsselte und signierte Recovery-Quelle; jede Wiederherstellung erzeugt eine neue signierte Restore-Operation.

Erlaubte Arbeiten

Der Agent darf:

- `trustedSnapshotService` implementieren.
- Snapshot-Struktur speichern.
- Snapshot-Signatur prüfen.
- Snapshot-Retention implementieren.
- Restore-Payloads erzeugen.
- RestoreFromSnapshot-Operationen bauen.
- Kategorie-Restore implementieren.
- Item-Restore implementieren.
- Tests für Snapshot-Manipulation schreiben.

Verbotene Arbeiten

Der Agent darf nicht:

- Snapshot als globale Wahrheit verwenden.
- automatische Rebaseline implementieren.
- Remote automatisch überschreiben.

- UI produktiv umbauen.
- alte Snapshot-Logik löschen, sofern Migration noch nicht fertig ist.

Sicherheitsregeln

- Snapshot wird lokal verschlüsselt.
- Snapshot wird signiert.
- Snapshot darf nur aus verified Records erstellt werden.
- Restore erzeugt immer neue signierte Operation.
- Kein direkter Upsert aus Snapshot.
- Manipulierter Snapshot darf nicht verwendet werden.

Erwartete Tests

Mindestens:

- Snapshot aus verified Records erzeugen.
- Snapshot-Signatur prüfen.
- manipulierte Snapshot-Daten werden abgelehnt.
- Restore erzeugt neue signierte Operation.
- Restore verwendet aktuelle AAD/RecordVersion.
- Kategorie-Restore hebt containerQuarantined nur auf, wenn Items selbst verified sind.

Akzeptanzkriterien

Phase 6 ist nur fertig, wenn:

- Trusted Snapshot Service existiert,
- Restore als signierte Operation funktioniert,
- keine automatische Rebaseline existiert,
- kein direkter Upsert aus Snapshot existiert,
- Tests Manipulation und Recovery abdecken.

Review-Fragen für dich

1. Wird ein Snapshot jemals als globale Wahrheit verwendet?
2. Wird Restore immer als neue signierte Operation erzeugt?
3. Gibt es irgendwo direkten Upsert aus Snapshot?
4. Gibt es Tests für manipulierte Snapshots?

Phase 7 — Migration vom alten System in neue Records und Operationen

Ziel

Phase 7 migriert bestehende Legacy-Daten sicher in das neue Operation-Log-Modell.

Diese Phase ist besonders kritisch, weil hier echte Nutzerdaten betroffen sein können.

Hauptziel in einem Satz

Baue eine nicht-destruktive, wiederaufnehmbare Migration, die alte Items und Kategorien in neue Records und initiale signierte Operationen überführt, ohne teilweise Migrationen als normalen Vault anzuzeigen.

Erlaubte Arbeiten

Der Agent darf:

- `migrationService` implementieren.
- Pre-Migration-Snapshot erstellen.
- Legacy-Daten lesen.
- Legacy-Daten entschlüsseln und schema-validieren.
- nicht migrierbare Daten als Legacy-Quarantäne markieren.
- neue Records erzeugen.
- initiale Create-Operationen erzeugen.
- erstes Device initialisieren.
- Migration-State speichern.
- Rollback/Retry-Pfade implementieren.
- Tests für vollständige und fehlgeschlagene Migration schreiben.

Verbotene Arbeiten

Der Agent darf nicht:

- alte Daten vor erfolgreicher Verifikation löschen.
- teilweise Migration als normalen Vault anzeigen.
- alte Runtime-Legacy-Bypässe offen lassen.
- Migration automatisch ohne Sicherheitsfreeze starten.
- echte Daten ohne Pre-Migration-Snapshot verändern.

Sicherheitsregeln

- Migration ist nicht-destruktiv.
- Vor Migration Snapshot.
- Server-Commit nur über neue RPCs.
- Kein direkter Tabellen-Upsert.
- Nach Commit muss der neue Zustand erneut geladen und verifiziert werden.
- Erst nach erfolgreicher Verifikation darf Legacy als migriert markiert werden.

Erwartete Tests

Mindestens:

- erfolgreiche Migration von Items.
- erfolgreiche Migration von Kategorien.

- Kategorie-ID landet im verschlüsselten Item-Plaintext.
- nicht entschlüsselbares Legacy-Item wird Legacy-Quarantäne.
- Fehler während Commit führt nicht zu normalem neuen Vault.
- Retry nach Fehler möglich.
- Pre-Migration-Snapshot bleibt erhalten.

Akzeptanzkriterien

Phase 7 ist nur fertig, wenn:

- Migration vollständig getestet ist,
- teilweise Migrationen sicher blockiert sind,
- Legacy-Pfade nur im Migration-Service existieren,
- keine Produktiv-UI neue Daten ohne Verifikation nutzt,
- klare Rollback-/Retry-Strategie dokumentiert ist.

Review-Fragen für dich

1. Was passiert, wenn Migration bei 50 Prozent abbricht?
2. Werden alte Daten vorher gelöscht?
3. Gibt es einen Pre-Migration-Snapshot?
4. Sind nicht migrierbare Items sicher quarantänisiert?
5. Wird nach Migration neu geladen und verifiziert?

Phase 8 — Shadow Mode und Parallelprüfung ohne UI-Umschaltung

Ziel

Phase 8 führt das neue System im Hintergrund aus, ohne die normale UI zu ersetzen. Das neue System prüft parallel, ob es denselben Vault-Zustand sicher verifizieren kann.

Hauptziel in einem Satz

Führe das neue Operation-Log-System im Shadow Mode aus, sammle Diagnose und Vergleichsergebnisse, aber verwende die Ergebnisse noch nicht für UI, Autofill, Export oder Suche.

Erlaubte Arbeiten

Der Agent darf:

- Shadow-Read aktivieren.
- neue State Machine im Hintergrund laufen lassen.
- Vergleich alter/neuer Zustände ohne Klartextlogs erstellen.
- Diagnose ohne Secrets speichern.

- Feature-Flag einbauen.
- Tests für Shadow Mode schreiben.

Verbotene Arbeiten

Der Agent darf nicht:

- UI auf neue Daten umschalten.
- Autofill umschalten.
- Export umschalten.
- Suche umschalten.
- alte Sicherheitslogik entfernen.
- Remote-Daten automatisch reparieren.

Sicherheitsregeln

- Diagnose enthält keine Klartexte.
- Shadow Mode darf keine produktiven Entscheidungen überschreiben.
- Shadow Mode darf keine Baseline neu setzen.
- Shadow Mode darf keine Daten löschen.

Erwartete Tests

Mindestens:

- Shadow Mode läuft ohne UI-Änderung.
- Shadow-Ergebnis beeinflusst Autofill nicht.
- Shadow-Ergebnis beeinflusst Export nicht.
- Diagnose enthält keine Secrets.
- Fehler im Shadow Mode sperrt nicht alten Produktivpfad.

Akzeptanzkriterien

Phase 8 ist nur fertig, wenn:

- neues System parallel prüft,
- keine Produktiventscheidung davon abhängt,
- Diagnose sicher ist,
- Feature-Flag funktioniert.

Review-Fragen für dich

1. Wird irgendeine UI aus Shadow-Ergebnissen gespeist?
 2. Kann Shadow Mode den Vault sperren?
 3. Enthält Diagnose Klartexte?
 4. Ist alles per Feature-Flag abschaltbar?
-

Phase 9 — UI-, Quarantäne-, Safe-Mode- und Konflikt-Anbindung hinter Feature-Flag

Ziel

Phase 9 bindet die neue State Machine an sichtbare UI-Zustände an, aber zunächst nur hinter Feature-Flag.

Hauptziel in einem Satz

Zeige verified, conflict, quarantine, safeMode und lockedCritical korrekt in der UI an, ohne Autofill/Export/Suche unsicher freizugeben.

Erlaubte Arbeiten

Der Agent darf:

- neue Vault Security Modes anzeigen.
- Quarantäne-Bereich an neue Gründe anbinden.
- Konflikt-UI anbinden.
- Safe-Mode-Anzeige anbinden.
- Kategorie-Quarantäne anzeigen.
- Restore/Delete/Resolve-Aktionen als signierte Operationen auslösen.
- Tests für UI-Zustände schreiben.

Verbotene Arbeiten

Der Agent darf nicht:

- Feature-Flag entfernen.
- Quarantäne-Records entschlüsseln.
- generische Accept-Aktion implementieren.
- Autofill für containerQuarantined freigeben, wenn erste Version konservativ bleiben soll.
- alte Logik bereits löschen, wenn Migration/Shadow Mode nicht stabil sind.

Sicherheitsregeln

- Normale Liste zeigt nur verified Records.
- Quarantäne zeigt keine unsicheren Plaintexts.
- Konflikte sind nicht Quarantäne.
- Kategoriefehler blockieren nicht den gesamten Vault.
- lockedCritical nur bei Root-Problemen.
- Jede UI-Aktion erzeugt konkrete signierte Operation.

Erwartete Tests

Mindestens:

- verified Item erscheint normal.
- unknown-author Item erscheint nur in Quarantäne ohne Plaintext.
- manipulated Item erscheint nicht normal.
- Kategorie manipuliert → Kategoriequarantäne, Vault nicht locked.
- conflict → Konfliktanzeige, nicht Quarantäne.
- RestoreFromSnapshot erzeugt Restore-Operation.
- DeleteUntrustedRemoteRecord erzeugt Delete-Operation.
- keine generische Accept-Aktion.

Akzeptanzkriterien

Phase 9 ist nur fertig, wenn:

- UI-Zustände korrekt getrennt sind,
- Quarantäne keine Plaintexts leakt,
- Aktionen signierte Operationen erzeugen,
- Feature-Flag weiterhin schützt,
- Tests alte Fehler abdecken.

Review-Fragen für dich

1. Kann ein quarantänisiertes Item irgendwo normal angezeigt werden?
 2. Gibt es noch einen Button namens Accept?
 3. Wird Kategoriefehler als Vault-Sperre behandelt?
 4. Welche Operation erzeugt jede UI-Aktion?
-

Phase 10 — Autofill, Export, Suche und Clipboard absichern

Ziel

Phase 10 bindet sicherheitskritische Datenabflüsse an die neuen Record-Zustände.

Hauptziel in einem Satz

Stelle sicher, dass Autofill, Export, Suche und Clipboard ausschließlich vollständig verified Records verwenden.

Erlaubte Arbeiten

Der Agent darf:

- Autofill-Gates an `verified` anbinden.
- Export-Gates an `verified` anbinden.
- Suchindex nur aus verified Plaintexts bauen.
- Clipboard nur aus verified Items erlauben.
- Klartext-State bei Quarantänewechsel invalidieren.
- Tests für alle Datenabflusswege schreiben.

Verbotene Arbeiten

Der Agent darf nicht:

- Quarantäne-Records autofüllen.
- Quarantäne-Records exportieren.
- Unknown-author Records durchsuchen.
- PendingVerification Records anzeigen.
- containerQuarantined für Autofill freigeben, sofern konservative erste Version gilt.

Sicherheitsregeln

- Autofill nur verified.
- Export nur verified.
- Suche nur verified.
- Clipboard nur verified.
- Quarantänewechsel invalidiert Klartext.

Erwartete Tests

Mindestens:

- quarantinedTampered erreicht Autofill nicht.
- quarantinedUnknownAuthor erreicht Export nicht.
- pendingVerification erreicht Suche nicht.
- Quarantänewechsel entfernt Suchindex.
- Quarantänewechsel deaktiviert Clipboard.
- verified Item funktioniert normal.

Akzeptanzkriterien

Phase 10 ist nur fertig, wenn:

- alle Datenabflusswege gated sind,
- Tests negativ und positiv abdecken,
- keine unsicheren Plaintexts in globalen Stores bleiben.

Review-Fragen für dich

1. Welche Zustände dürfen Autofill nutzen?
 2. Welche Zustände dürfen exportiert werden?
 3. Was passiert mit Suchindex bei Quarantäne?
 4. Was passiert mit Clipboard bei Quarantäne?
-

Phase 11 — Alte Runtime-Logik entfernen

Ziel

Phase 11 entfernt alte Sicherheitslogik aus Produktivpfaden, nachdem das neue System getestet, migriert und hinter Feature-Flag stabil ist.

Hauptziel in einem Satz

Entferne alte Rebaseline-, Snapshot-Digest-, TTL- und Kategorie-Globalblockade-Logik aus Runtime-Pfaden, ohne Migration-Recovery zu zerstören.

Erlaubte Arbeiten

Der Agent darf:

- alte automatische Rebaseline entfernen.
- alte Snapshot-Digest-Vertrauensentscheidung entfernen.
- `LOCAL_WRITE_CACHE_TTL_MS` entfernen.
- Kategorie-Globalblockade entfernen.
- direkte alte Vault-Writes entfernen.
- alte Tests umschreiben oder löschen, wenn sie altes Verhalten erwarten.
- Legacy-Code auf Migration-Service begrenzen.

Verbotene Arbeiten

Der Agent darf nicht:

- Migration-Service zerstören.
- Legacy-Daten unlesbar machen, bevor Migration abgeschlossen ist.
- neue Sicherheitsschleusen entfernen.
- Feature-Flag zu früh entfernen, falls Rollout noch nicht abgeschlossen ist.

Sicherheitsregeln

- Keine alte Runtime-Sicherheitsentscheidung darf parallel weiterleben.
- Legacy-Code nur Migration-Service.
- Keine auskommentierten Altpfade.
- Keine Compatibility-Bypässe.
- Tests dürfen altes Verhalten nicht mehr als korrekt bestätigen.

Erwartete Tests

Mindestens:

- keine automatische Rebaseline bei unbekannter Remote-Änderung.
- Kategoriefehler blockieren nicht Vault.
- TTL-Trust existiert nicht mehr.
- direkte Upserts auf alte Vault-Tabellen existieren nicht mehr in Runtime.
- alte Tests, die Rebaseline erwarten, wurden entfernt oder umgeschrieben.

Akzeptanzkriterien

Phase 11 ist nur fertig, wenn:

- alte Runtime-Sicherheitslogik entfernt ist,
- keine toten Altpfade existieren,
- keine Legacy-Flags alte Logik reaktivieren,
- Migration-Service eng begrenzt bleibt,
- Regressionstests neue Regeln beweisen.

Review-Fragen für dich

1. Welche alten Dateien/Pfade wurden entfernt?
2. Welche alten Tests wurden geändert und warum?
3. Gibt es noch `LOCAL_WRITE_CACHE_TTL_MS` ?
4. Gibt es noch automatische Rebaseline?
5. Gibt es noch Kategorie-Globalblockade?

Phase 12 — Rollout, echte Integrationstests und Abschlussreview

Ziel

Phase 12 macht aus dem neuen System einen freigabefähigen Produktivpfad.

Hauptziel in einem Satz

Führe das neue System kontrolliert aus dem Feature-Flag heraus, validiere echte DB-/Runtime-Flows und dokumentiere Restrisiken.

Erlaubte Arbeiten

Der Agent darf:

- Feature-Flag-Rollout vorbereiten.
- echte Supabase-Local-Integrationstests ausführen.

- Web/Tauri-Runtime testen.
- Multi-Client-Szenarien testen.
- Offline-/Online-Sync testen.
- Migration End-to-End testen.
- finale Security-Doku schreiben.

Verbotene Arbeiten

Der Agent darf nicht:

- Tests ignorieren.
- bekannte Sicherheitsfehler als Restrisiko wegdokumentieren, wenn sie das Zielmodell verletzen.
- Feature-Flag entfernen, bevor Rollback-Plan existiert.

Erwartete Tests

Mindestens:

- Supabase-local RLS/RPC-Tests.
- Web ändert Item, Tauri integriert ohne Quarantäne.
- Tauri ändert Kategorie, Web blockiert Vault nicht.
- Server manipuliert Ciphertext, Record wird nicht entschlüsselt.
- Server löscht Record ohne Delete, Restore möglich.
- zwei Clients offline ändern dasselbe Item, Conflict entsteht.
- Migration E2E erfolgreich.
- Migration-Abbruch sicher.
- Autofill/Export/Suche/Clipboard nur verified.

Akzeptanzkriterien

Phase 12 ist nur fertig, wenn:

- alle kritischen Tests bestanden sind,
- bekannte pre-existing Tests getrennt dokumentiert sind,
- Rollback-Plan existiert,
- Restrisiken dokumentiert sind,
- Architektur und Implementierung konsistent sind,
- keine alten Sicherheitslogiken parallel leben.

Review-Fragen für dich

1. Welche echten Runtime-Flows wurden getestet?
2. Welche Tests sind noch rot und warum?
3. Gibt es einen Rollback-Plan?
4. Sind alle alten Runtime-Pfade entfernt?
5. Ist das System ohne Feature-Flag produktionsreif?

3. Standard-Prompt-Vorlage für jede Phase

Diese Vorlage kannst du verwenden, wenn du einem KI-Modell eine Phase gibst.

Du arbeitest im Repository von Singra Vault auf dem Branch feature/vault-operation-log-quarantine-integrity.

Arbeite ausschließlich an PHASE X: <Phasenname>.

Das Architekturkonzept bleibt verbindlich. Ändere das Sicherheitskonzept nicht, außer du dokumentierst einen klaren Widerspruch und stoppst zur Rückfrage.

Verboten:

- keine Arbeiten außerhalb dieser Phase,
- keine UI-Anbindung, sofern diese Phase es nicht ausdrücklich erlaubt,
- keine Autofill-/Export-/Such-/Clipboard-Änderungen, sofern diese Phase es nicht ausdrücklich erlaubt,
- keine Migration, sofern diese Phase es nicht ausdrücklich erlaubt,
- keine Entfernung alter Runtime-Logik, sofern diese Phase es nicht ausdrücklich erlaubt,
- keine direkten Vault-Tabellenwrites außerhalb der Operation-Schicht,
- keine Secrets loggen,
- keine Tests löschen, um grün zu werden.

Vor Beginn:

1. Prüfe Branch und Git-Status.
2. Liste die Dateien auf, die du voraussichtlich änderst.
3. Liste die Sicherheitsinvarianten auf, die diese Phase betrifft.
4. Stoppe, wenn uncommitted Änderungen existieren, die nicht zu dieser Phase gehören.

Während der Umsetzung:

- kleine, nachvollziehbare Commits,
- Tests direkt mitimplementieren,
- keine stillen Bypässe,
- keine auskommentierten Altpfade,
- keine neue Dependency ohne Begründung.

Am Ende:

1. Nenne geänderte Dateien.
2. Nenne neue oder geänderte Services.
3. Nenne entfernte oder bewusst nicht entfernte Altpfade.
4. Nenne Tests mit Ergebnis.
5. Nenne bekannte pre-existing Failures getrennt.
6. Nenne nicht verifizierte Annahmen.
7. Nenne Risiken und nächste Phase.

4. Konkreter Prompt für Phase 3

Du arbeitest im Repository von Singra Vault auf dem Branch `feature/vault-operation-log-quarantine-integrity`.

Arbeite ausschließlich an Phase 3: Repository-Layer und Feature-Flag, ohne UI.

Ziel:

Baue einen isolierten `vaultOpLog-Repository-/RPC-Layer` für die neuen Supabase-RPCs aus Phase 2. Der neue Layer darf noch nicht produktiv von UI, `VaultContext`, `Autofill`, `Export`, `Suche` oder `Clipboard` verwendet werden.

Vor Beginn prüfen:

1. Branch und Git-Status.
2. Existenz der Phase-1-Module unter `src/services/vaultOpLog/`.
3. Existenz der Phase-2-Migrationen und RPCs.
4. Ob `previous_ciphertext_hash`, `intent_id`, `rebased_from_op_id` und `bootstrap_vault_trust` vorhanden sind.
5. Ob uncommitted Änderungen existieren.

Erlaubt:

- Repository-Service für `submit_vault_operation`.
- Repository-Service für `get_vault_head`.
- Repository-Service für `get_vault_changes_since`.
- Repository-Service für `get_vault_records_by_ids`.
- Repository-Service für `bootstrap_vault_trust`.
- Typisierte Mapper zwischen Supabase-JSON/DB und Domain-Typen.
- Feature-Flag für neue `vaultOpLog`-Flows.
- Tests für Mapper, JSONB/canonical roundtrip, `op_hash`-Stabilität, idempotente RPC-Ergebnisse und Konfliktklassifikation.

Verboten:

- keine UI-Änderungen.
- keine `VaultContext`-Produktivanbindung.
- keine `Autofill`-Änderungen.
- keine `Export`-Änderungen.
- keine `Suchindex`-Änderungen.
- keine `Clipboard`-Änderungen.
- keine Migration alter Nutzerdaten.
- keine Entfernung alter Runtime-Logik.
- keine direkten Vault-Tabellenwrites.
- keine Feature-Flag-Umgehung.

Sicherheitsregeln:

- `signed_body` aus DB ist nicht automatisch vertrauenswürdig.
- Der Client muss `canonicalize` erneut ausführen.
- `op_hash` muss nach Roundtrip stabil bleiben.
- `previous_ciphertext_hash` muss vollständig erhalten bleiben.
- `intent_id` und `rebased_from_op_id` müssen vollständig erhalten bleiben.

- base_vault_head und resulting_vault_head müssen vollständig erhalten bleiben.
- Repository-Layer trifft keine endgültige Sicherheitsentscheidung; das gehört in Phase 5 zur State Machine.

Tests mindestens:

- Mapper behalten alle sicherheitsrelevanten Felder.
- signed_body survives JSONB-like roundtrip.
- op_hash bleibt stabil nach Mapping.
- gleicher op_id + gleicher op_hash wird als idempotent klassifiziert.
- gleicher op_id + anderer op_hash wird als Fehler klassifiziert.
- stale base_vault_head wird als rebase-needed klassifiziert.
- stale previous_ciphertext_hash wird als record-conflict klassifiziert.
- bootstrap_vault_trust ist nur als Initialisierungspfad verfügbar.

Am Ende ausgeben:

- Branch-Name.
- Geänderte Dateien.
- Neue Services.
- Welche Produktivpfade nicht berührt wurden.
- Tests mit Ergebnis.
- Bekannte pre-existing Failures.
- Nicht verifizierte Annahmen.
- Risiken für Phase 4.

5. Verbraucher-Übersicht: Welche Phase macht was?

Phase	Thema	Darf Produktivpfade ändern?	Risiko	Status
0	ADR, Inventar, Architekturentscheidung	Nein	niedrig	abgeschlossen
1	Crypto/AAD/Hash/Signing/Trust-Module	Nein	niedrig-mittel	abgeschlossen
2	DB-Schema, RLS, RPCs	Nein	mittel	abgeschlossen + gehärtet
3	Repository-Layer + Feature-Flag	Nein	mittel	nächster Schritt
4	Pending Queue + Operation Builder + Rebase	Nein	mittel-hoch	offen
5	State Machine + Verifikationspipeline	Noch nicht UI	hoch	offen

Phase	Thema	Darf Produktivpfade ändern?	Risiko	Status
6	Trusted Snapshots + Recovery	Noch nicht UI	hoch	offen
7	Legacy-Migration	Ja, aber kontrolliert	sehr hoch	offen
8	Shadow Mode	Nur parallel	hoch	offen
9	UI/Quarantäne/Safe Mode hinter Flag	Ja, hinter Flag	hoch	offen
10	Autofill/Export/Suche/ Clipboard Gates	Ja	sehr hoch	offen
11	Alte Runtime-Logik entfernen	Ja	sehr hoch	offen
12	Rollout + E2E + Abschlussreview	Ja	sehr hoch	offen

6. Stop-Regeln für KI-Agenten

Ein KI-Agent muss stoppen und Rückfrage stellen, wenn:

1. er außerhalb der aktuellen Phase arbeiten müsste,
2. er alte Runtime-Logik entfernen müsste, obwohl die Phase das nicht erlaubt,
3. er UI/Autofill/Export/Suche/Clipboard ändern müsste, obwohl die Phase das nicht erlaubt,
4. er eine neue Dependency für Kryptografie vorschlägt,
5. er Secrets oder echte Nutzerdaten sieht,
6. er Migrationen ausführen müsste, obwohl die Phase das nicht erlaubt,
7. er Tests löschen müsste, um grün zu werden,
8. er das Sicherheitskonzept ändern müsste,
9. er nicht erklären kann, welche signierte Operation eine Nutzeraktion erzeugt,
10. er nicht beweisen kann, dass manipulierte Records nicht entschlüsselt werden.

7. Wichtigste Kontrollfrage vor jeder nächsten Phase

Vor jeder Phase fragst du:

Ist die vorherige Phase abgeschlossen, getestet, committed und in ihrer Review-Zusammenfassung verständlich dokumentiert?

Wenn nein:

Nicht weitergehen.

8. Aktueller empfohlener nächster Schritt

Nach dem gemeldeten Stand ist der nächste sinnvolle Schritt:

Phase 3 – Repository-Layer und Feature-Flag, ohne UI

Nicht Phase 4, nicht UI, nicht Migration.

Vor Phase 3 muss bestätigt sein:

- `previous_ciphertext_hash` ist überall korrekt statt `previous_record_hash`.
- CAS prüft gegen `vault_records.ciphertext_hash`.
- `intent_id` und `rebased_from_op_id` existieren.
- `bootstrap_vault_trust` existiert.
- SECURITY DEFINER RPCs haben festen `search_path`.
- JSONB-Roundtrip-Tests sind grün.
- Phase-2-Vertragstests sind grün.

Wenn das bestätigt ist, kann Phase 3 gestartet werden.